学　　号　：

202209502

上海立达学院

SHANGHAI LIDA UNIVERSITY

本科毕业论文（设计）

基于 LightGBM 的金融交易欺诈检测系统设计与实现

学院名称　：　　数字科学学院

专业名称：数据科学与大数据技术

学生姓名：　　王迎汀

指导教师：

2026　年　3　月　20　日

BACHELOR'S DEGREE THESIS OF SHANGHAI LIDA UNIVERSITY

Design and Implementation of a Financial Transaction Fraud Detection System Based on LightGBM

Candidate：Wang Yingting

Supervisor：

March 20, 2026

郑重声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所

取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果由本人承担。

毕业论文（设计）成果归上海立达学院所有。

作者专业　　：　数据科学与大数据技术

作者学号　　：　202209502

作者签名　　：

年　月　日（手填时间）

基于 LightGBM 的金融交易欺诈检测系统设计与实现

摘　要

针对金融交易场景中欺诈样本占比极低、误报成本高、处置时效要求严格等问题，本文以 OpenML 公开信用卡交易数据集为基础，设计并实现了一套基于 LightGBM 的金融交易欺诈检测系统。系统围绕“离线训练评估、在线接口推理、准实时流式监控、可视化界面支撑”四条主线展开，在数据层面采用时间有序切分方式构建训练集、验证集和测试集，在模型层面对逻辑回归、随机森林、LightGBM 基线、下采样 LightGBM 和加权 LightGBM 等方案进行对比，并在固定误报率约束下执行阈值搜索。实验结果表明，本文最终选取的 LightGBM 基线方案在测试集上取得 0.7729 的 PR-AUC、0.8649 的召回率和 0.0063 的误报率，在极度不平衡场景下兼顾了识别能力与业务可接受性。在工程实现方面，本文基于 FastAPI 封装预测服务，基于 Streamlit 构建可视化系统界面，并使用 Kafka 与 Spark Structured Streaming 搭建准实时评分链路，实现了风险打分、批量预测、监控展示和论文配图导出。研究结果表明，面向表格型金融交易数据，采用以 LightGBM 为核心、以阈值优化为补充、以流式处理为延展的整体方案，能够较好满足本科毕业论文研究场景下对准确性、可解释性和系统完整性的综合要求。

关键词：金融交易欺诈检测；极度不平衡数据；LightGBM；阈值优化；流式处理

BACHELOR'S DEGREE THESIS OF SHANGHAI LIDA UNIVERSITY

ABSTRACT

To address the challenges of highly imbalanced class distribution, costly false alarms, and strict response requirements in financial transactions, this paper designs and implements a financial transaction fraud detection system based on LightGBM with the public OpenML credit card dataset as the experimental foundation. The system is organized around four coordinated parts: offline training and evaluation, online inference service, near-real-time streaming monitoring, and dashboard-based visualization. A temporal split strategy is adopted to build the training, validation, and test sets, while Logistic Regression, Random Forest, baseline LightGBM, undersampled LightGBM, and weighted LightGBM are compared under a threshold optimization mechanism constrained by a fixed false positive rate. The selected baseline LightGBM model achieves a PR-AUC of 0.7729, a recall of 0.8649, and a false positive rate of 0.0063 on the test set, showing a practical balance between detection effectiveness and business tolerance in an extremely imbalanced setting. From the engineering perspective, FastAPI is used to expose prediction interfaces, Streamlit is employed to build the presentation dashboard, and Kafka together with Spark Structured Streaming is used to construct a near-real-time scoring pipeline. The completed system supports risk scoring, batch prediction, monitoring, and thesis-oriented asset export. The study demonstrates that an integrated scheme centered on LightGBM, enhanced by threshold optimization and extended by streaming processing, is suitable for the design and implementation of a financial fraud detection system at the undergraduate thesis level.

Key words: financial transaction fraud detection; highly imbalanced data; LightGBM; threshold optimization; streaming processing

目　录

1 绪　论 1
1.1 研究背景与研究意义 1
1.2 国内外研究现状 1
1.3 研究内容与论文创新点 2
1.4 论文结构安排 3
2 相关技术与理论基础 4
2.1 极度不平衡分类问题 4
2.2 LightGBM 模型原理 4
2.3 评价指标与阈值优化方法 4
2.4 系统实现相关技术 5
3 需求分析与数据资源设计 6
3.1 业务场景与系统目标 6
3.2 数据资源分析 6
3.3 功能输出与接口设计要求 7
4 系统设计 9
4.1 系统总体架构设计 9
4.2 离线训练模块设计 10
4.3 服务层与接口模块设计 10
4.4 准实时处理链路设计 11
4.5 可视化界面设计 12
5 系统实现与关键技术 13
5.1 数据准备与预处理实现 13
5.2 模型训练与阈值优化实现 13
5.3 在线推理服务实现 14
5.4 准实时评分与监控实现 15
5.5 图表与论文素材导出实现 18
6 测试与结果分析 19
6.1 实验环境与测试方案 19
6.2 最优模型效果分析 20
6.3 系统功能测试分析 25
7 结论与展望 26

7.1 研究结论 26
7.2 存在的不足 26
7.3 后续展望 26
参考文献 27
致 谢 29

1 绪　论

1.1 研究背景与研究意义

随着移动支付、线上消费和互联网金融服务的持续普及，金融交易规模呈现高频、海量和跨渠道并发的特征。在交易效率不断提升的同时，欺诈攻击的组织化、自动化和隐蔽化程度也显著增强，金融机构既要在毫秒级完成风险判断，又要尽量避免对正常交易造成不必要拦截。传统以人工经验为核心的规则库虽然部署门槛较低，但面对多变的欺诈模式时往往表现出维护成本高、泛化能力弱和误报率难以控制等问题[1][4]。

金融交易欺诈检测的难点首先来自样本分布的极端不均衡。以信用卡交易为代表的真实业务数据中，正常交易通常占比超过 99%，欺诈样本仅占极少比例。若仍以总体准确率作为目标，模型容易偏向多数类，进而形成“准确率很高但欺诈识别能力很弱”的伪优结果。He 和 Garcia 指出，不平衡分类问题必须从数据层、算法层和评价层同步建模，否则模型难以在少数类上获得稳定收益[2]。因此，如何在低误报约束下提升欺诈召回率，成为该领域最具现实意义的核心问题之一。

从工程应用角度看，欺诈检测不仅是一个分类模型问题，更是一个系统问题。模型训练、阈值设定、接口服务、流式接入、监控可视化和结果解释共同决定了系统的可用性。如果只有离线实验结果而缺少服务化封装与运行链路，就难以支撑真实场景中的接入、联调和演示；如果只有实时链路而缺少可靠的离线评估机制，则又难以证明模型选择的合理性。因此，毕业设计需要在算法效果和系统完整性之间建立平衡，使论文既体现数据科学方法论，也体现软件工程实现能力。

1.2 国内外研究现状

现有金融欺诈检测研究大体经历了三个阶段。第一阶段以规则引擎和专家系统为主，重点依赖业务人员定义高风险规则，例如短时间异地消费、同卡高频小额试探、异常终端登录等。这一类方法具备较强可解释性，但对新型欺诈模式的适应速度较慢。Bhattacharyya 等通过对多种欺诈检测方法的对比指出，单一规则方法很难同时兼顾鲁棒性与可迁移性[4]。

第二阶段研究重点转向监督学习模型。逻辑回归、决策树、随机森林、支持向量机等方法被广泛用于从历史交易中学习分类边界。Whitrow 等提出交易聚合思

路，通过在单笔交易之外构造时间窗口统计量，提升模型对异常行为的感知能力[5]；Randhawa 等则利用集成学习提升模型鲁棒性，说明在信用卡欺诈问题中，多模型投票能够改善识别性能[7]。然而，随着数据规模上升和不平衡程度加剧，传统模型在训练效率、少数类召回和误报约束控制方面逐渐暴露局限。

第三阶段研究更强调“极度不平衡建模 + 高性能树模型 + 在线化部署”的融合路线。Chawla 等提出的 SMOTE 为少数类过采样提供了经典解决思路[3]；Dal Pozzolo 等从实践视角总结了信用卡欺诈检测的关键经验，强调时间切分、数据泄漏防控和运营成本敏感性[1][13]；Ke 等提出的 LightGBM 通过基于梯度的一侧采样和互斥特征捆绑技术显著提高了表格数据训练效率，使梯度提升树在大规模结构化数据场景中具备更强实用价值[6]。在实时检测方向，Carcillo 等将流式数据处理与活动学习结合，验证了持续更新机制在真实信用卡场景中的应用潜力[8]。

总体来看，国内外研究已经形成较为清晰的方法谱系，但仍存在两个值得继续深入的问题。其一，许多论文着重于算法效果，却缺少面向实际部署与运行验证的系统闭环；其二，不少工作使用静态测试指标论证模型性能，却未进一步说明阈值设定与业务容忍误报率之间的关系。因此，围绕真实公开数据构建一套可运行、可验证、可复现实验结果的欺诈检测系统，仍具有较强的教学与实践价值。

1.3 研究内容与论文创新点

本文以“金融交易欺诈检测系统设计与实现”为研究对象，围绕数据准备、模型训练、阈值优化、服务封装、准实时监控和可视化展示六个方面展开。在数据层面，本文使用 OpenML 公开信用卡交易数据集，基于时间字段执行顺序切分，避免未来信息泄漏到训练阶段；在模型层面，构建逻辑回归、随机森林与多种 LightGBM 方案的对比实验；在评价层面，以 PR-AUC、召回率、FPR 和 Recall@Fixed-FPR 作为核心指标；在系统层面，则进一步将最优模型封装为 FastAPI 服务，并接入 Kafka 与 Spark Structured Streaming 完成准实时演示链路。

本文的主要工作与特点体现在以下三个方面。第一，围绕极度不平衡问题给出可落地的实验流程，既比较不同模型，也显式执行固定误报率约束下的阈值搜索，使模型选择更贴近业务目标。第二，在离线实验之外完成了在线服务、批量推理、准实时监控看板和论文配图导出模块，提升了系统完整性。第三，论文写作全过程以项目源码、运行产物和截图为依据，尽量避免脱离实现细节的空泛描述，使系统分析与实现论证保持一致。

1.4 论文结构安排

全文共分为七章。第一章说明研究背景、研究现状、研究内容和论文结构。第二章介绍极度不平衡分类、LightGBM、评价指标以及系统所使用的工程技术。第三章从业务场景出发，对系统需求和数据资源进行分析。第四章给出系统总体架构和各功能模块设计。第五章结合关键源码说明系统实现过程。第六章展示模型实验结果、系统测试情况和运行效果分析。第七章总结全文并提出后续优化方向。

2 相关技术与理论基础

2.1 极度不平衡分类问题

极度不平衡分类问题是指目标类别中少数类样本数量远小于多数类样本数量，且模型训练目标又主要关注少数类识别质量的场景。在金融欺诈检测中，欺诈交易比例往往低于 1%，模型若仅依赖总体准确率，将更倾向于输出多数类。He 和 Garcia 将不平衡学习方法概括为数据层重采样、算法层代价敏感学习和评价层指标改造三条主要路线[2]。在本课题中，这三条路线分别对应随机下采样与 SMOTE 思路、类别权重约束以及 PR-AUC 与 FPR 受限召回指标。

对欺诈检测而言，漏报和误报的代价并不对称。漏报意味着真实欺诈交易被放行，会直接造成资产损失；误报则意味着正常交易被拦截，可能引起客户流失和人工复核成本上升。因此，模型优化目标不能简单理解为“把预测正确的样本尽量变多”，而应转化为“在误报率可接受的前提下尽可能发现更多欺诈交易”。这一目标导向决定了本文在阈值设定上采用固定 FPR 约束，而不是默认使用 0.5 概率阈值。

2.2 LightGBM 模型原理

LightGBM 属于梯度提升决策树家族。与传统提升树相比，其在工程层面引入了基于梯度的一侧采样和互斥特征捆绑两项关键优化。前者优先保留梯度较大的样本以估计信息增益，降低扫描成本；后者将互斥稀疏特征压缩至同一特征桶中，从而减少特征维度开销[6]。由于信用卡交易数据以数值型特征为主，且训练规模较大，LightGBM 能够在训练速度和分类性能之间取得较好平衡。

本项目中，LightGBM 被分别用于基线方案、下采样方案和加权方案。基线方案直接在原始不平衡数据上训练；下采样方案通过 RandomUnderSampler 压缩多数类规模；加权方案则通过设置正负样本比例权重，提高模型对少数类的关注度。通过并行比较这三种方案，可以更清楚地观察不同不平衡处理策略对 PR-AUC、召回率和误报率的影响。

2.3 评价指标与阈值优化方法

针对极度不平衡场景，本文重点采用精确率、召回率、误报率、F1 值、PR-AUC

以及 Recall@Fixed-FPR 等指标。其中，PR-AUC 能够更灵敏地反映少数类识别质量；Recall@Fixed-FPR 用于评价在固定误报上限下模型还能保留多少召回能力。为便于后续分析，本文将核心指标的定义写为如下形式。

$$Precision = \frac{TP}{TP + FP} \tag{2.1}$$

$$Recall = \frac{TP}{TP + FN} \tag{2.2}$$

$$FPR = \frac{FP}{FP + TN} \tag{2.3}$$

式中，TP 表示真正例数，FP 表示假正例数，FN 表示假负例数，TN 表示真负例数。在本系统实现中，阈值搜索并不是直接固定 0.5，而是遍历模型输出分数的分位点集合，优先选择满足 FPR 不超过 0.01 的候选阈值，再在这些候选值中比较召回率和精确率表现，从而得到更符合业务要求的决策边界。

2.4 系统实现相关技术

在算法之外，本系统还使用了若干关键工程组件。FastAPI 用于暴露健康检查、指标查询、特征模式和预测接口，具备类型校验清晰、异步框架兼容性好等特点。Streamlit 用于构建可视化系统界面，可在较短时间内完成数据图表、指标卡片和交互控件展示。Spark Structured Streaming 则承担准实时批次处理角色，使系统能够在本地环境中模拟交易流接入、模型服务调用和结果落盘过程[9][10]。

这些组件的引入使本课题不再停留于离线模型实验，而是形成了较为完整的“训练—服务—流式处理—可视化”闭环。对于毕业设计而言，这种闭环既有利于展示系统实现能力，也有利于将论文图表、截图和实验结果直接来自项目运行产物，提升论文证据链的可信度。

3 需求分析与数据资源设计

3.1 业务场景与系统目标

本系统面向信用卡交易欺诈识别场景，目标是在极度不平衡数据条件下构建一个兼具实验评估、在线推理和准实时处理能力的研究型系统。从业务视角看，系统应能对单笔交易或批量交易输出欺诈概率、风险等级和解释性线索；从研究视角看，系统应能支持多模型对比、阈值优化、实验结果导出和过程可视化；从论文支撑视角看，系统应具备界面截图、图表资产和流式监控结果。

表 3.1 系统核心需求分析

需求类别	具体要求
离线训练	支持多模型实验、阈值搜索、指标导出和图表生成
在线推理	支持单笔预测、批量预测、模式查询和健康检查
准实时监控	支持 Kafka 消息回放、Spark 微批评分与结果落盘
可视化展示	支持总览、预测、监控和论文配图查看
论文支撑	输出截图、图表、模型文件和实验统计结果

根据上述需求，系统的主体对象包括离线训练模块、模型文件与指标产物、API 推理服务、流式评分任务、结果存储目录以及前端展示页面。各对象之间通过文件、HTTP 接口和消息流形成协作关系。这意味着系统设计不仅要考虑单一模块的正确性，还要考虑模块间输入输出协议是否稳定，以及本地环境下的可复现性。

3.2 数据资源分析

项目核心训练数据来自 OpenML 公开的信用卡交易数据集。该数据集包含 2013 年 9 月欧洲信用卡交易记录，共 284807 条样本，其中欺诈交易 492 条，欺诈比例仅为 0.1727%[14]。数据集中除 Time、Amount 等原始字段外，还包含经过匿名化处理的 V1 至 V28 主成分特征，目标列为 Class。由于特征已经过脱敏与数值化处理，数据较适合作为表格型欺诈检测研究的标准实验样本。

表 3.2 训练数据集基本信息

项目	数值
样本总数	284807

特征列数	30
目标字段	Class
欺诈样本数	492
欺诈样本占比	0.1727%
数据来源	OpenML creditcard

为了避免随机分层切分导致未来信息提前泄漏到训练集，本文优先采用基于 Time 字段的时间顺序切分。在代码实现上，训练集、验证集与测试集按照 70%、15%、15%的比例从前至后划分，再将训练阶段样本上限裁剪至 120000 条以降低本地迭代成本。这一做法与 Dal Pozzolo 等关于信用卡欺诈时序建模的实践经验保持一致，即实验设计要尽量贴近真实部署中的时间推进过程[1][13]。

表 3.3 数据集划分结果

数据集	样本数	欺诈样本数	欺诈占比
train	120000	344	0.2867%
valid	42721	74	0.1732%
test	42722	74	0.1732%

3.3 功能输出与接口设计要求

从输入输出关系看，系统需要支持三类主要请求。第一类为离线训练请求，输入为原始数据集和实验配置，输出为模型文件、指标 JSON、实验对比 CSV 和图表 PNG。第二类为在线预测请求，输入为单笔或批量交易特征，输出为欺诈概率、风险等级和解释原因。第三类为流式监控请求，输入为 Kafka 消息队列中的交易记录，输出为最新批次结果、汇总统计和可视化页面展示。为了保持模块解耦，系统通过统一的特征模式定义约束各模块之间的数据字段。

表 3.4 API 接口设计

接口	方法	功能说明
/health	GET	返回服务状态与模型加载状态
/metrics	GET	返回最佳实验指标
/schema	GET	返回模型所需特征模式
/predict	POST	执行单笔交易欺诈预测
/predict/batch	POST	执行批量交易欺诈预测

除功能需求外，系统还应满足若干非功能要求。其一是可复现性，即模型训练与论文配图能够通过脚本重复执行；其二是可读性，即指标、表格和截图能够直接用于论文撰写；其三是稳定性，即当模型文件不存在或输入特征缺失时，接口能够给出明确报错；其四是可视化表达能力，即页面布局需要突出模型指标、运

行流程与案例结果。这些要求共同决定了后续系统架构与实现方案。

4 系统设计

4.1 系统总体架构设计

基于需求分析，本文将系统划分为数据层、模型层、服务层和表示层四个层次。数据层负责原始数据下载、清洗和切分；模型层负责多组实验训练、阈值优化和指标生成；服务层负责模型加载、特征校验和接口调用；表示层负责界面渲染、批量预测结果呈现和流式结果监控。各层之间通过标准化目录与接口进行连接，从而保证离线训练、在线预测和准实时处理能够围绕同一套模型产物协同运行。

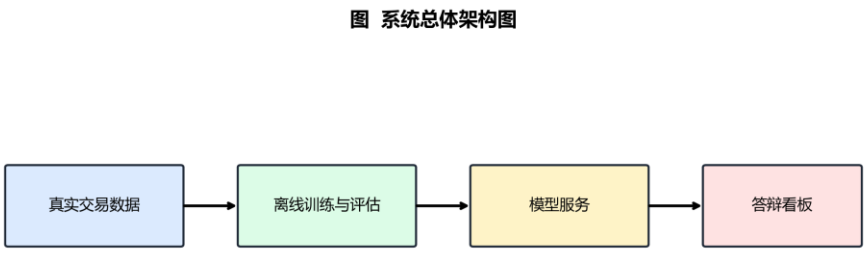

图 4.1 系统总体架构图

如图 4.1 所示，系统首先从真实公开交易数据出发，经由离线训练与评估模块生成最优模型和实验指标；之后通过模型服务对外提供预测能力，并进一步由可视化界面汇总呈现各类结果。该架构突出了本文的主线逻辑：先论证模型有效，再论证系统可用。

4.2 离线训练模块设计

离线训练模块由数据读取、数据切分、实验配置、模型训练、阈值搜索、指标保存和图表导出七个子过程组成。训练脚本首先读取本地信用卡数据，并依据 split_data 函数输出训练集、验证集和测试集；随后遍历预定义实验列表，对逻辑回归、随机森林以及不同 LightGBM 策略分别训练；每组实验都会在验证集上寻找满足误报率约束的最优阈值，再将阈值应用至测试集评估。这种先在验证集定阈值、再在测试集报告结果的流程，有助于避免将阈值直接拟合到测试集上。

训练模块的输出不仅包含数值指标，还包含模型文件和论文图表。例如 experiment_results.csv 用于汇总各方案对比结果，best_metrics.json 用于供接口和页面读取，best_model.joblib 用于保存最优模型及其特征摘要，图表目录则存储 PR 曲线、ROC 曲线、阈值权衡图、混淆矩阵和特征重要性图。从论文写作角度看，这些产物直接构成了第六章实验分析的证据来源。

4.3 服务层与接口模块设计

服务层采用 FastAPI 实现，核心目标是将离线训练得到的模型包装成稳定、可复用的 HTTP 接口。健康检查接口用于确认服务进程是否正常；指标接口用于让

前端页面获取当前最佳模型表现；特征模式接口用于前端和流式任务生成输入模板；预测接口则用于输出欺诈评分和风险解释。在预测过程中，服务会首先检查必需特征是否齐全，缺失字段将触发 400 异常，从而避免不完整输入直接进入模型。

为了提升结果可解释性，系统没有简单返回 0 或 1 分类结果，而是额外返回阈值、风险等级和偏离度最大的前三个特征。其中，风险等级由 score_to_risk_level 函数根据模型分数与阈值关系分为 high、medium 和 low 三个层级；解释原因由 summarize_reasons 函数结合训练集特征中位数和四分位距计算而得。这种轻量级解释机制虽然不等同于严格的 SHAP 分析，但足以说明模型为何将某笔交易判断为高风险。

4.4 准实时处理链路设计

为了展示系统在流式场景下的扩展能力，本文在本地环境中设计了 Kafka 与 Spark Structured Streaming 的准实时链路。交易回放脚本将测试集样本逐条写入 Kafka 主题，Spark 流式任务从主题中读取数据后转换为 DataFrame，再调用 API 批量预测接口完成打分，最后将结果写入本地 JSONL 和 CSV 文件，并同步刷新汇总统计。由于本课题的重点仍是欺诈检测整体流程而非高并发流处理性能，因此流式任务采取微批方式实现，既保证实现复杂度可控，也能够直观展示数据流转过程。

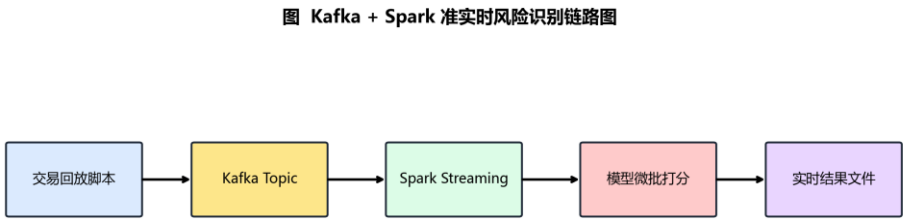

图 4.2 准实时风险识别链路图

图 4.2 展示了准实时链路中的五个关键环节：交易回放脚本负责模拟外部数据输入，Kafka 负责消息缓冲，Spark Streaming 负责微批处理与接口调用，模型微批打分负责输出风险结果，最终结果文件则作为 Streamlit 监控页面的数据来源。这一设计使项目具备了从静态实验走向动态展示的能力。

4.5 可视化界面设计

表示层以 Streamlit 为基础，划分为系统总览、离线预测、准实时监控和论文配图四个页签。系统总览页集中展示最佳模型、PR-AUC、召回率、误报率、阈值和训练摘要；离线预测页用于查看测试集单条样本及批量 CSV 预测结果；准实时监控页用于展示最新批次号、事件数量、平均风险分和流式曲线；论文配图页则集中管理图表和截图资源。这种布局既服务于系统调试，也为论文截图整理与运行结果核验提供了统一入口。

5 系统实现与关键技术

5.1 数据准备与预处理实现

在实现层面，项目首先通过 download_creditcard_dataset 函数完成 OpenML 数据集抓取与本地缓存。该函数会自动检查数据列中是否存在 Class 目标字段，并在下载完成后生成对应元数据文件，用于记录样本规模、欺诈样本数量与数据来源。从可追溯性角度看，这一步既满足训练需要，也为论文中的数据资源说明提供了直接依据。

预处理阶段的核心函数为 split_data。当数据中存在 Time 字段时，函数会按时间顺序对数据进行切分；若用户显式选择分层方案，则退化为 train_test_split 实现的分层抽样。这一设计保留了实验灵活性，同时默认采用更符合业务场景的时间切分。此外，预处理模块还封装了 undersample 与 smote 两类采样器接口，为后续实验配置提供统一入口。

5.2 模型训练与阈值优化实现

模型训练主流程定义在 train_model.py 与 modeling.py 中。训练脚本首先构造实验列表，再调用 run_experiment 逐个完成模型拟合、验证集阈值搜索和测试集评估。每次实验结束后，系统都会实时写出部分实验结果，避免长时间运行导致中间结果丢失。在 LightGBM 配置上，本文选择 250 棵树、0.05 学习率、63 个叶节点和 0.9 子采样比例等参数，兼顾训练效率与拟合能力。

代码 5.1 固定误报率约束下的阈值搜索逻辑

```
def optimize_threshold(y_true, scores, max_fpr=0.01):
    thresholds = np.unique(np.concatenate(([0.0], np.quantile(scores,
np.linspace(0.0, 1.0, 513)), [1.0])))
    best_threshold = 0.5
    best_snapshot = None
    for threshold in thresholds:
        predictions = (scores >= threshold).astype(int)
        fpr = false_positive_rate(y_true, predictions)
        precision = precision_score(y_true, predictions, zero_division=0)
        recall = recall_score(y_true, predictions, zero_division=0)
        f1 = f1_score(y_true, predictions, zero_division=0)
        if fpr <= max_fpr:
```

```
            snapshot = {"precision": precision, "recall": recall, "f1": f1,
"fpr": fpr}
            if best_snapshot is None or recall > best_snapshot["recall"]:
                best_threshold = float(threshold)
                best_snapshot = snapshot
    return best_threshold, best_snapshot
```

代码 5.1 展示了阈值搜索的核心思想。系统并不假设默认概率阈值合理，而是遍历分位点构成的候选阈值集合，并筛选出满足最大误报率限制的结果。当多个阈值都满足 FPR 约束时，系统优先选择召回率更高的方案，若召回率相同再比较精确率。这使模型训练过程能够更贴近金融反欺诈“宁可少放过真实欺诈，也不能让误报无限增加”的现实要求。

5.3 在线推理服务实现

在线推理服务的核心在于统一模型输入、输出协议。系统使用 predict_records 函数接收前端或流式任务传入的交易记录列表，并将其重组为按照 feature_columns 顺序排列的 DataFrame。如果输入中缺少必需特征，则立即抛出异常，避免模型在字段错位时产生不可控结果。完成概率预测后，系统进一步计算风险等级和主要偏离特征，最终返回结构化 JSON。

代码 5.2 单笔交易预测接口实现

```
@app.post('/predict')
def predict(payload: PredictRequest):
    bundle = _load_model_bundle()
    if bundle is None:
        raise HTTPException(status_code=404, detail='Model not found. Train
the model first.')
    record = {'transaction_id': payload.transaction_id,
**payload.features}
    try:
        return predict_records(bundle, [record])[0]
    except ValueError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
```

代码 5.2 给出了单笔预测接口的实现方式。接口首先尝试从缓存中加载模型包，若模型尚未训练则返回 404 错误；在模型存在的前提下，接口将请求中的交易编号和特征字典合并为标准记录，再调用 predict_records 函数输出预测结果。该接口设计具有良好的前后端解耦特征，既可服务 Streamlit 页面，也可供批处理与流处理任务复用。

图 5.1 系统总览页面运行截图

图 5.1 所示的系统总览页面集中呈现了系统运行的核心信息。页面顶部显示最佳模型、主要指标和最优阈值，中部展示项目摘要、训练摘要和关键图表。与传统仅给出单张截图的论文不同，本系统的总览页能够同时承担系统入口与论文证据汇总界面的功能。

5.4 准实时评分与监控实现

准实时链路的实现重点在于让流式任务与模型服务保持松耦合。在 run_streaming_job.py 中，Spark 首先通过/schema 接口拉取特征列表，构建对应的结构化模式，随后从 Kafka 主题读取消息并解析为 DataFrame。在 foreachBatch 回调中，每一批数据会被转换为 Pandas 数据框，并通过/predict/batch 接口统一发送到模型服务。这样做的好处是：即使后续模型替换为其他算法，只要接口协议保持不变，流式任务逻辑仍可复用。

代码 5.3 Spark 批次评分与结果落盘逻辑

```
def process_batch(batch_df, batch_id: int) -> None:
    pdf = batch_df.toPandas()
    if pdf.empty:
        return
    merged = score_batch_via_api(pdf, args.api_base_url)
    summary = persist_stream_results(merged, STREAMING_DIR,
batch_id=batch_id)
```

```
    print(json.dumps(summary, ensure_ascii=False))
```

代码 5.3 说明了流式任务的最小处理闭环。当批次为空时函数直接返回；当批次存在内容时，系统调用 API 完成打分，再将结果交给 persist_stream_results 进行持久化。该函数会同时维护 scored_events.jsonl、latest_summary.json 和 latest_batch.csv 三个产物文件，前者用于保留完整事件流，后两者用于支撑实时页面显示。

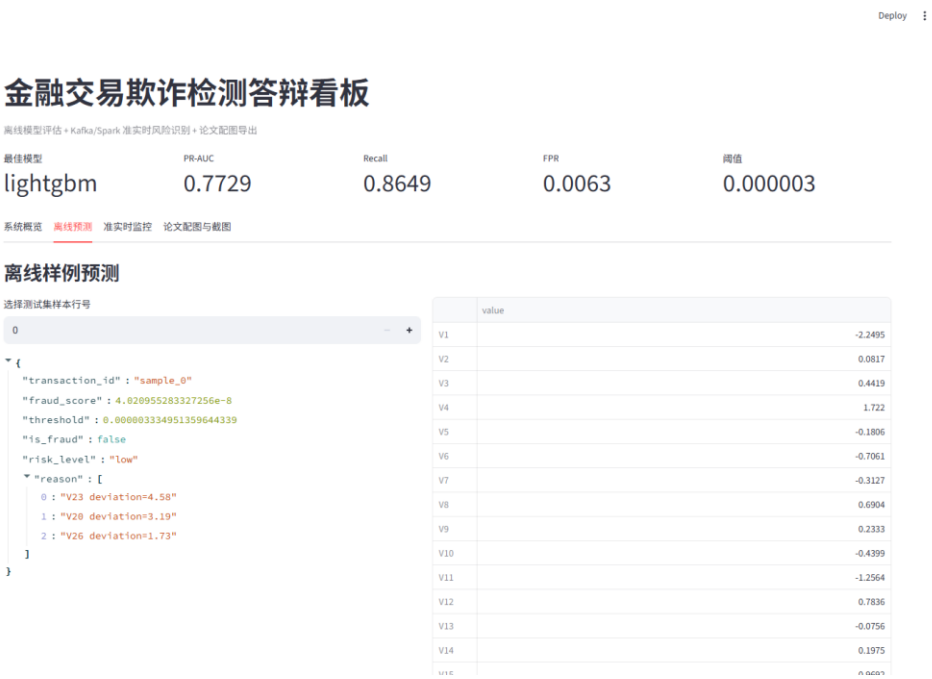

图 5.2 离线样例预测页面截图

图 5.3 准实时监控页面截图

图 5.2 展示了离线样例预测页面，页面左侧直接给出交易样本的风险分、阈值、是否判定为欺诈以及偏离最明显的特征；图 5.3 展示了准实时监控页面，可观察最新批次编号、批次事件数、平均风险分以及最近一批结果的走势。二者共同证明，项目不仅完成了模型训练，而且已经实现了以模型为核心的应用化外壳。

5.5 图表与论文素材导出实现

为了降低论文撰写过程中的素材整理成本，项目额外提供 export_thesis_assets.py 与 capture_dashboard_screenshots.py 两个辅助脚本。前者根据实验结果自动生成系统架构图、流式链路图和模型对比图，并输出论文配图清单；后者调用 Playwright 自动打开系统界面并捕获固定页面截图。这一设计使论文中的大部分图片都能由代码稳定复现，而不是依赖手工截图和后期拼接。

6 测试与结果分析

6.1 实验环境与测试方案

实验环境部署于 Windows 本地开发机，项目主要基于 Python 3.11、scikit-learn、LightGBM、FastAPI、Streamlit、Kafka 与 Spark Structured Streaming 实现。在模型实验部分，系统使用训练集进行模型拟合，验证集进行阈值确定，测试集用于最终报告指标。在系统测试部分，则从接口可用性、页面展示完整性、批量预测能力和准实时链路可运行性四个维度进行验证。此外，项目还包含若干 pytest 测试文件，用于覆盖指标计算、API 接口和页面展示相关逻辑。

表 6.1 不同实验方案对比结果

实验方案	PR-AUC	Precision	Recall	F1	FPR
lightgbm_baseline	0.7729	0.1922	0.8649	0.3145	0.0063
rf_weighted	0.7513	0.1441	0.8784	0.2476	0.0091
logreg_baseline	0.7057	0.2342	0.8514	0.3673	0.0048
lightgbm_undersample	0.6481	0.4091	0.8514	0.5526	0.0021
lightgbm_weighted	0.4687	0.0183	0.9189	0.0359	0.0855

由表 6.1 可见，基线 LightGBM 方案在 PR-AUC 指标上取得最佳结果，为 0.7729；随机森林加权方案虽然在召回率上略高，但误报率也更高；下采样 LightGBM 方案在精确率与 F1 值方面更优，却牺牲了整体 PR-AUC；加权 LightGBM 由于阈值极端化，导致精确率明显下降。由此可以说明，在本数据集和本参数配置下，直接使用 LightGBM 基线并进行阈值优化，是兼顾综合效果与实现复杂度的最优选择。

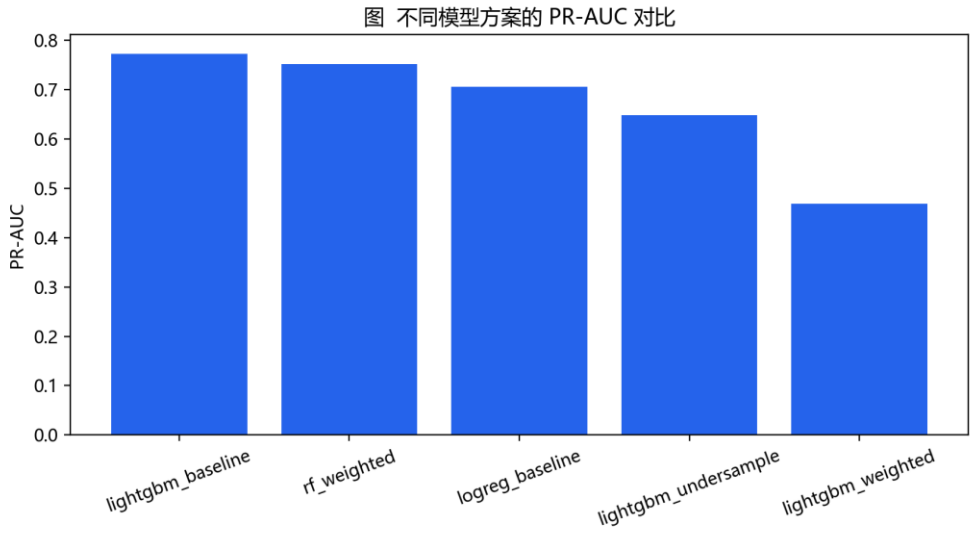

图 6.1 不同模型方案的 PR-AUC 对比

图 6.1 以柱状图形式呈现各实验方案的 PR-AUC 差异。从图中可以清楚地看到，LightGBM 基线与随机森林加权方案明显优于逻辑回归，而加权 LightGBM 表现最差。这一结果说明，仅仅增加类别权重并不能保证收益，阈值选择、模型结构和样本分布之间仍需协同优化。

6.2 最优模型效果分析

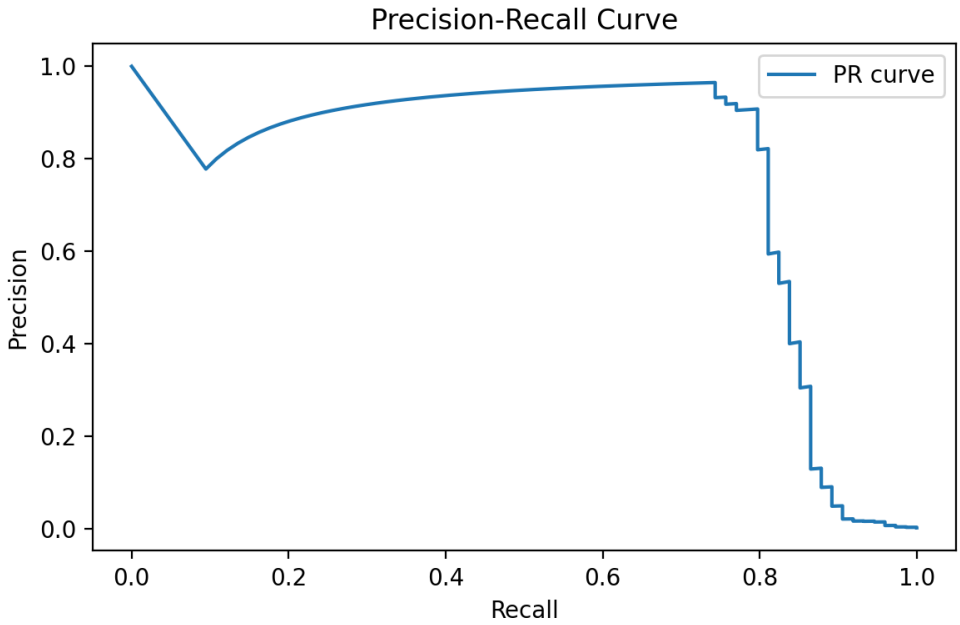

图 6.2 最优模型 PR 曲线

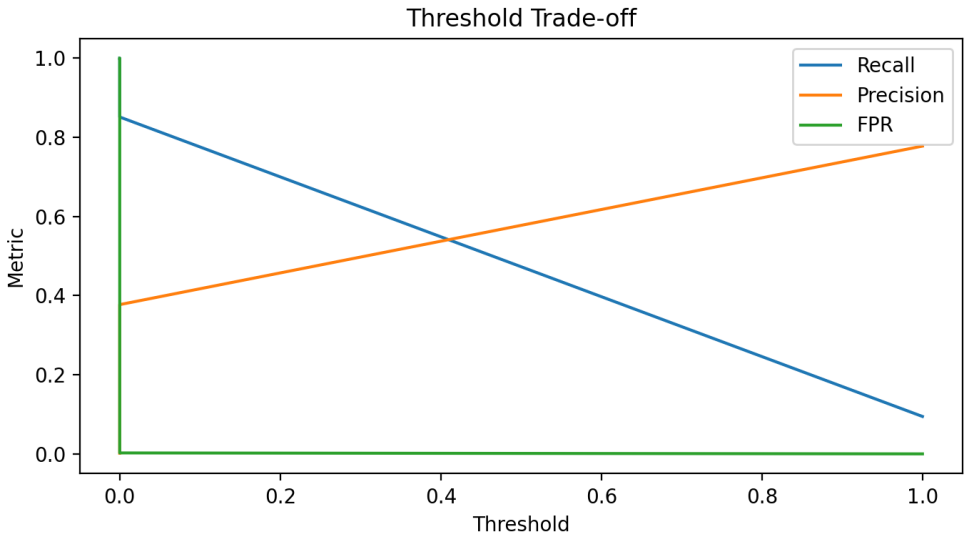
Threshold Trade-off
Recall
Precision
FPR
Metric
Threshold
0.0
0.2
0.4
0.6
0.8
1.0

图 6.3 阈值权衡曲线

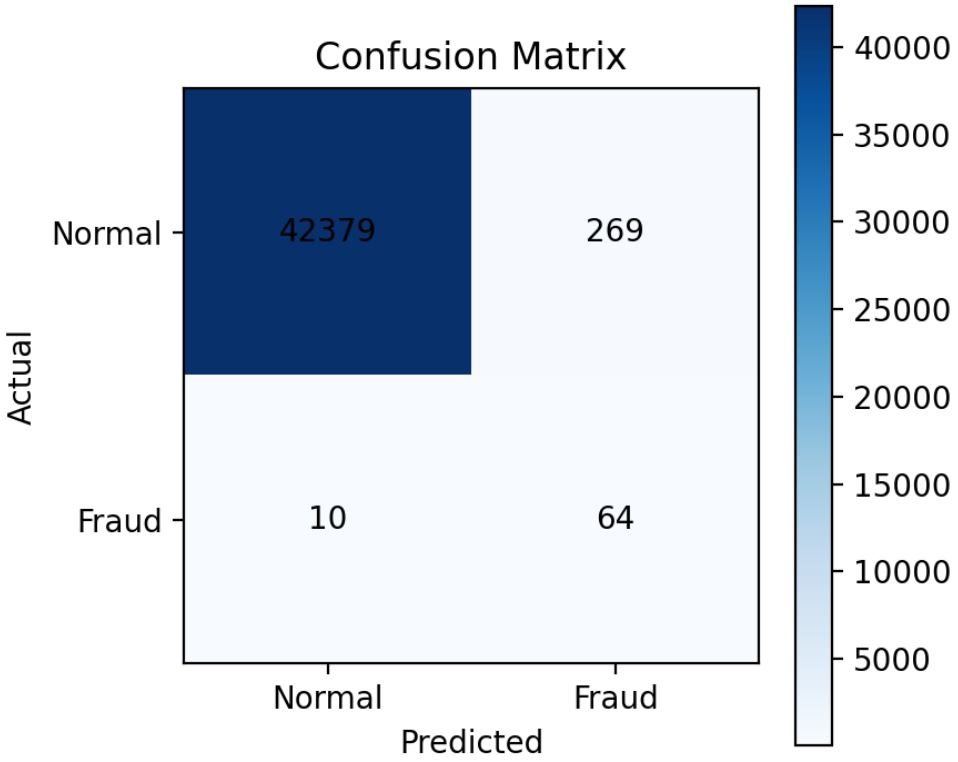

图 6.4 最优模型混淆矩阵

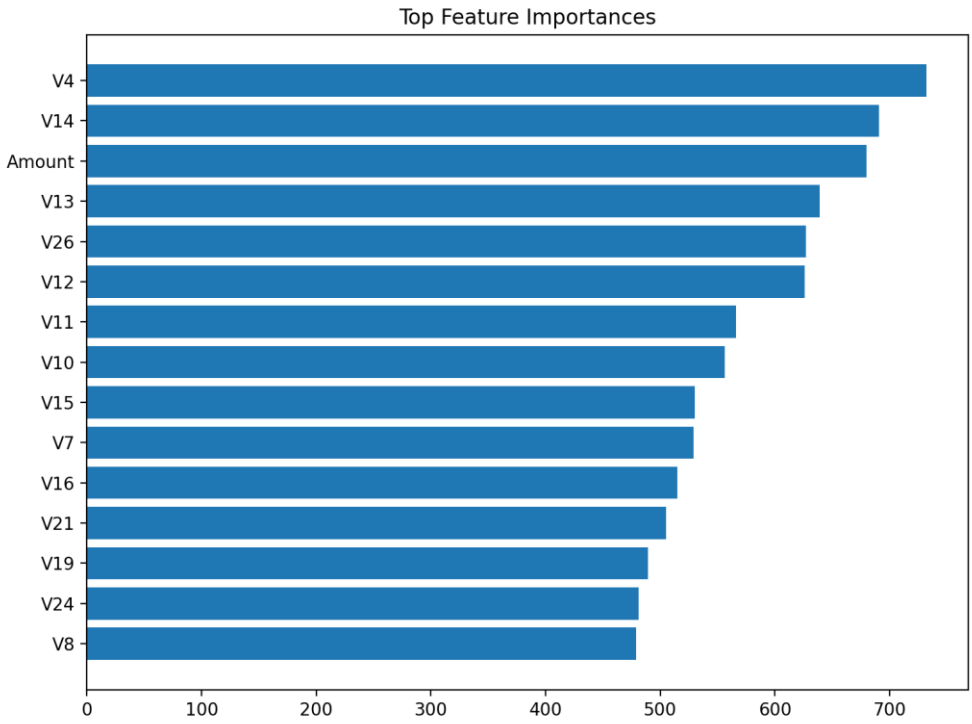

图 6.5 最优模型特征重要性排序

在最优模型分析中，PR 曲线、阈值权衡图、混淆矩阵和特征重要性图共同构成了完整证据链。图 6.2 说明最优模型在高召回区域仍能保持较好的精确率曲线形态，证明其并未完全依赖极低阈值换取召回。图 6.3 则反映出随着阈值升高，召回率下降、精确率上升、误报率下降的典型权衡关系，这也验证了阈值优化的必要性。

根据测试集混淆矩阵，系统在 42722 条测试样本中识别出 64 条真实欺诈，同时产生 269 条误报，漏报 10 条。从业务视角看，这一结果意味着系统已经能够发现大多数欺诈交易，但仍需要借助后续规则复核或人工审核控制误报成本。图 6.5 展示了特征重要性排序结果，其中 V4、V14、Amount、V13 和 V26 等特征对最终决策贡献更大。由于原始数据经过匿名化处理，论文无法给出这些特征的业务语义解释，但排序结果仍能证明模型并非随机决策，而是围绕少数关键维度建立了稳定的分类边界。

表 6.2 最优模型前十重要特征

特征名	重要性
V4	732
V14	691
Amount	680
V13	639

V26	627
V12	626
V11	566
V10	556
V15	530
V7	529

6.3 系统功能测试分析

表 6.3 系统功能测试结果

测试项	测试方式	预期结果	测试结论
健康检查接口	GET /health	返回 status=ok	通过
指标查询接口	GET /metrics	返回 PR-AUC 等指标	通过
特征模式接口	GET /schema	返回 29 个特征及示例值	通过
单笔预测功能	离线样例页面	输出风险分、阈值和解释原因	通过
批量预测功能	上传 CSV 文件	生成批量预测结果表	通过
准实时监控功能	Kafka+Spark 回放	生成最新批次汇总和监控图	通过
自动化测试	pytest	接口与展示逻辑断言成功	通过

系统功能测试表明，健康检查、指标查询、特征模式查询、单笔预测、批量预测、看板展示和流式结果落盘等功能均能按预期运行。其中，pytest 测试覆盖了指标计算函数、API 接口、站点内容与展示逻辑等模块。这说明项目不仅具备可运行结果，也具备一定程度的回归验证能力。

需要指出的是，准实时链路目前仍属于本地验证型实现。其一，系统采用微批方式调用 HTTP 接口，而非真正的嵌入式在线特征服务；其二，当前数据源仍来自测试集回放，而非持续接入的真实业务流；其三，风险解释采用特征偏离度近似实现，尚未引入更严格的特征贡献分析。但从本文研究目标来看，这样的实现已经能够较完整地说明模型服务化与准实时扩展路径。

7 结论与展望

7.1 研究结论

本文围绕极度不平衡金融交易欺诈检测问题，完成了一套基于 LightGBM 的金融交易欺诈检测系统设计与实现。论文从研究背景、技术基础、需求分析、系统设计、关键实现和测试分析等方面展开论证，并基于真实公开信用卡交易数据完成多模型对比实验。结果表明，采用时间切分、LightGBM 建模和固定误报率约束下的阈值优化，能够在测试集上取得较好的综合表现。同时，FastAPI、Streamlit、Kafka 与 Spark Structured Streaming 的组合也使系统具备了较完整的服务与展示能力。

7.2 存在的不足

受项目周期与计算资源限制，本文仍存在若干不足。首先，实验数据主要依赖单一公开数据集，尚未纳入更多跨机构、跨时段的真实数据进行外部验证；其次，特征工程主要沿用了匿名化原始字段，并未在业务语义层面构造更丰富的聚合特征；再次，流式部分主要用于验证系统链路，尚未就吞吐量、延迟和故障恢复进行系统性压测。这些不足意味着本文结论更适用于教学研究与原型验证场景，而不是直接面向生产级金融风控部署。

7.3 后续展望

后续研究可以从三个方面继续深化。一是引入更丰富的行为序列特征、设备指纹特征和图结构关系特征，以提升对团伙型欺诈的识别能力；二是在解释层面融合 SHAP 值、局部规则提取等方法，为风险审核提供更强的可解释支持；三是在部署层面尝试引入容器化编排、在线特征仓与持续训练机制，真正形成可持续更新的反欺诈系统。如果在更大规模数据和更复杂业务环境下继续验证，本文提出的设计思路仍具有进一步扩展的潜力。

参考文献

[1] Dal Pozzolo A, Caelen O, Le Borgne Y A, et al. Learned lessons in credit card fraud detection from a practitioner perspective[J]. Expert Systems with Applications, 2014, 41(10): 4915-4928.

[2] He H, Garcia E A. Learning from Imbalanced Data[J]. IEEE Transactions on Knowledge and Data Engineering, 2009, 21(9): 1263-1284.

[3] Chawla N V, Bowyer K W, Hall L O, et al. SMOTE: Synthetic Minority Over-sampling Technique[J]. Journal of Artificial Intelligence Research, 2002, 16: 321-357.

[4] Bhattacharyya S, Jha S, Tharakunnel K, et al. Data mining for credit card fraud: A comparative study[J]. Decision Support Systems, 2011, 50(3): 602-613.

[5] Whitrow C, Hand D J, Juszczak P, et al. Transaction aggregation as a strategy for credit card fraud detection[J]. Data Mining and Knowledge Discovery, 2009, 18(1): 30-55.

[6] Ke G, Meng Q, Finley T, et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree[C]//Advances in Neural Information Processing Systems 30. Red Hook: Curran Associates, 2017.

[7] Randhawa K, Loo C K, Seera M, et al. Credit Card Fraud Detection Using AdaBoost and Majority Voting[J]. IEEE Access, 2018, 6: 14277-14284.

[8] Carcillo F, Le Borgne Y A, Caelen O, et al. Streaming active learning strategies for real-life credit card fraud detection: assessment and visualization[J]. International Journal of Data Science and Analytics, 2018, 5(4): 285-300.

[9] Zaharia M, Xin R S, Wendell P, et al. Apache Spark: A unified engine for big data processing[J]. Communications of the ACM, 2016, 59(11): 56-65.

[10] Apache Software Foundation. Apache Kafka Documentation[EB/OL]. https://kafka.apache.org/documentation/, 2026-03-20.

[11] Breiman L. Random Forests[J]. Machine Learning, 2001, 45(1): 5-32.

[12] Liu F T, Ting K M, Zhou Z H. Isolation Forest[C]//2008 Eighth IEEE International

Conference on Data Mining. Washington, DC: IEEE, 2008: 413-422.

[13] Dal Pozzolo A, Boracchi G, Caelen O, et al. Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy[J]. IEEE Transactions on Neural Networks and Learning Systems, 2018, 29(8): 3784-3797.

[14] OpenML. Credit Card Fraud Detection Dataset[DB/OL]. https://api.openml.org/d/42397, 2026-03-20.

致 谢

在本次毕业设计与论文撰写过程中，我得到了学院老师与同学们的大力帮助。首先，感谢指导教师在选题确定、系统实现思路和论文结构安排方面给予的悉心指导；其次，感谢数字科学学院各位任课教师在本科阶段所讲授的数据挖掘、机器学习、数据库和软件工程等课程内容，为本课题的完成奠定了扎实基础；再次，感谢在项目调试和资料整理过程中给予支持与建议的同学们。最后，感谢家人对我学习和生活上的理解与支持。谨以此文向所有关心和帮助过我的人表示诚挚谢意。